

Lab Assignment 6

Aim

To study and implement **database triggers and cursors** in SQL/PLSQL for automating tasks, enforcing constraints, and handling row-by-row operations.

Objectives

- To understand the concept and functionality of **triggers** in databases.
- To learn how triggers can be used for **automatic execution** of procedures on events like INSERT, UPDATE, or DELETE.
- To explore **cursors** for row-wise data manipulation and retrieval.
- To develop practical skills in writing **PL/SQL blocks** using triggers and cursors.
- To analyze how triggers and cursors improve **data integrity, consistency, and automation** in database systems.

Requirements

- A relational database management system (Oracle/MySQL/PostgreSQL).
- SQL/PLSQL environment (e.g., Oracle SQL*Plus, MySQL Workbench).
- Tables with sample data (e.g., Students, Courses, Enrollment).
- Knowledge of basic SQL commands (DDL, DML).
- Understanding of PL/SQL block structure (declarations, execution, exception handling).

Theory

Triggers

- A **trigger** is a stored procedure in a database that is automatically executed in response to certain events on a table or view.
- Types of triggers:
 - **Before Trigger:** Executes before an operation (e.g., before INSERT).
 - **After Trigger:** Executes after an operation (e.g., after UPDATE).
 - **Instead Of Trigger:** Used for views to replace the triggering action.
- Applications:
 - Enforcing business rules.
 - Maintaining audit logs.
 - Validating data before insertion.

- Preventing invalid transactions.

◆ **Cursors**

- A **cursor** is a pointer that allows row-by-row processing of query results.
- Types of cursors:
 - **Implicit Cursor**: Automatically created by SQL for single row queries.
 - **Explicit Cursor**: Declared by the programmer for handling multiple rows.
- Cursor operations:
 - **Declare** the cursor.
 - **Open** the cursor to execute the query.
 - **Fetch** rows one by one.
 - **Close** the cursor after processing.
- Applications:
 - Iterating through query results.
 - Performing calculations or updates row by row.
 - Handling complex business logic not possible with set-based SQL.

Code:

```
1) -- Create Students table

CREATE TABLE Students (
    student_id INT PRIMARY KEY,
    name VARCHAR(50),
    course VARCHAR(50)
);

-- Insert sample data

INSERT INTO Students VALUES (1, 'Alice', 'Computer Science');
INSERT INTO Students VALUES (2, 'Bod', 'Information Technology');
INSERT INTO Students VALUES (3, 'Charles', 'Electronics');

2) -- Create Student_Log table for auditing

CREATE TABLE Student_Log (
```

```
log_id INT PRIMARY KEY AUTO_INCREMENT,  
student_id INT,  
old_name VARCHAR(50),  
new_name VARCHAR(50),  
updated_on TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

3) delimiter @

```
CREATE TRIGGER trg_student_update  
AFTER UPDATE ON Students  
FOR EACH ROW  
BEGIN  
    INSERT INTO Student_Log(student_id, old_name, new_name)  
    VALUES (OLD.student_id, OLD.name, NEW.name);  
END@  
Delimiter ;
```

4) UPDATE Students

```
SET name = 'Aaron'  
WHERE student_id = 1;  
SELECT * FROM Student_Log;
```

5) DELIMITER \$\$

```
CREATE PROCEDURE ShowTotalMarks()  
BEGIN  
    DECLARE v_student_id INT;  
    DECLARE v_total_marks INT;  
    DECLARE done INT DEFAULT 0;
```

```

-- Declare cursor
DECLARE cur_marks CURSOR FOR
    SELECT student_id, SUM(marks) AS total
    FROM Marks
    GROUP BY student_id;

-- Declare handler for end of cursor
DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = 1;

-- Open cursor
OPEN cur_marks;

read_loop: LOOP
    FETCH cur_marks INTO v_student_id, v_total_marks;
    IF done = 1 THEN
        LEAVE read_loop;
    END IF;

    -- Output result (in MySQL, use SELECT instead of DBMS_OUTPUT)
    SELECT CONCAT('Student ID: ', v_student_id,
        ' | Total Marks: ', v_total_marks) AS Result;
END LOOP;

-- Close cursor
CLOSE cur_marks;

END$$

DELIMITER ;
6) CALL ShowTotalMarks();

```

Output:

Conclusion: